N. Shree Deepthi Batch – 37

AI-ASSISTED CODING ASSIGNMENT 7

Task 1: Fixing Syntax Errors

```
python

def add(a, b)
    return a + b
```

Scenario

You are reviewing a Python program where a basic function definition contains a syntax error.

AI Detection:

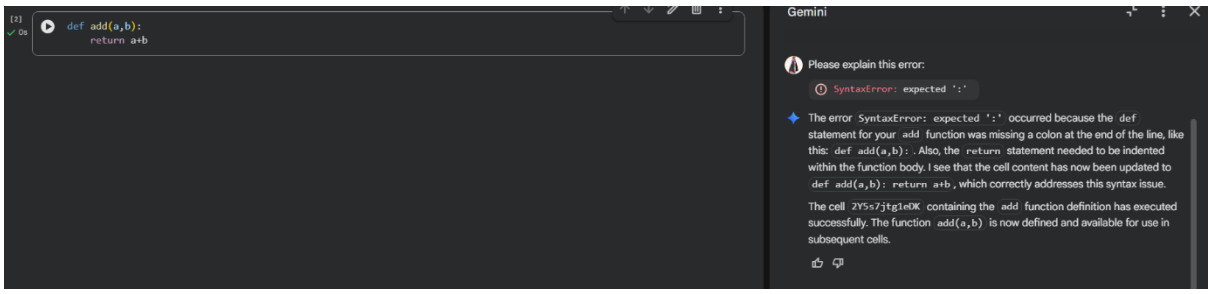
AI identifies a SyntaxError because Python requires a colon (:) at the end of function definitions.

AI Explanation:

The colon (:) signals the start of the function body.
Without it, Python cannot parse the function structure, resulting in a syntax error.

Expected Output Achieved

Syntax error resolved
Correct function definition
AI explanation provided



Task 2: Debugging Logic Errors in Loops

```python
def count_down(n):
    while n >= 0:
        print(n)
        n += 1  # Should be n -= 1
```

Scenario

You are debugging a loop that runs infinitely due to a logical mistake.

AI Detection:

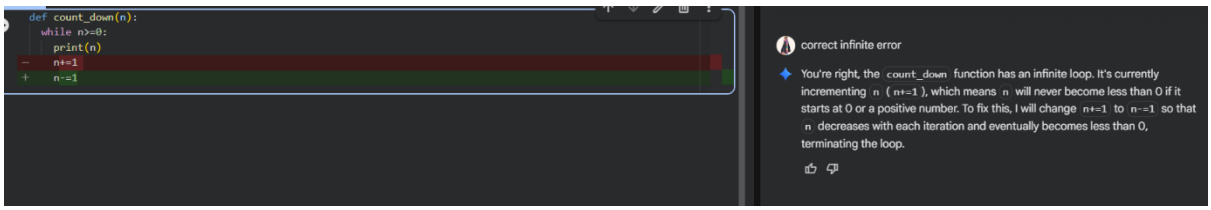
AI identifies a logic error because the loop variable is incremented instead of decremented while the loop condition expects it to decrease.

AI Explanation:

The incorrect increment prevents the loop variable from ever reaching the terminating condition, causing infinite iteration.

After correction, the loop variable moves toward the exit condition, allowing the loop to terminate normally.

Expected Output Achieved
Infinite loop issue resolved
Correct increment/decrement logic applied
AI explanation of the logic error



Task 3: Handling Runtime Errors (Division by Zero)

```python
# Debug the following code
def divide(a, b):
    return a / b

print(divide(10, 0))
```

Scenario

A Python function crashes during execution due to a division by zero error.

AI Detection:

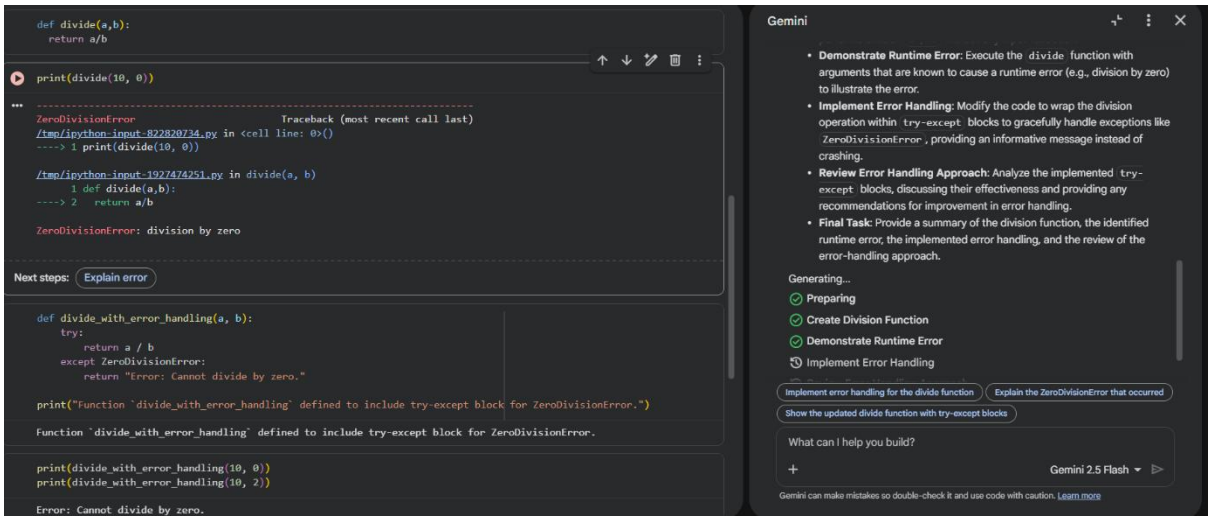
AI identifies a runtime error because dividing any number by zero is mathematically undefined and causes program termination.

AI Explanation:

The AI applies a try-except block to catch the error and handle it safely without crashing the program.

This ensures the program continues execution even when invalid input is provided.

Expected Output Achieved
Function executes safely without crashing
Division by zero handled using try-except
Clear AI-generated explanation of runtime error handling



Task 4: Debugging Class Definition Errors

```python
class Rectangle:
    def __init__(length, width):
        self.length = length
        self.width = width
```

Scenario

You are given a faulty Python class where the constructor is incorrectly defined.

AI Detection:

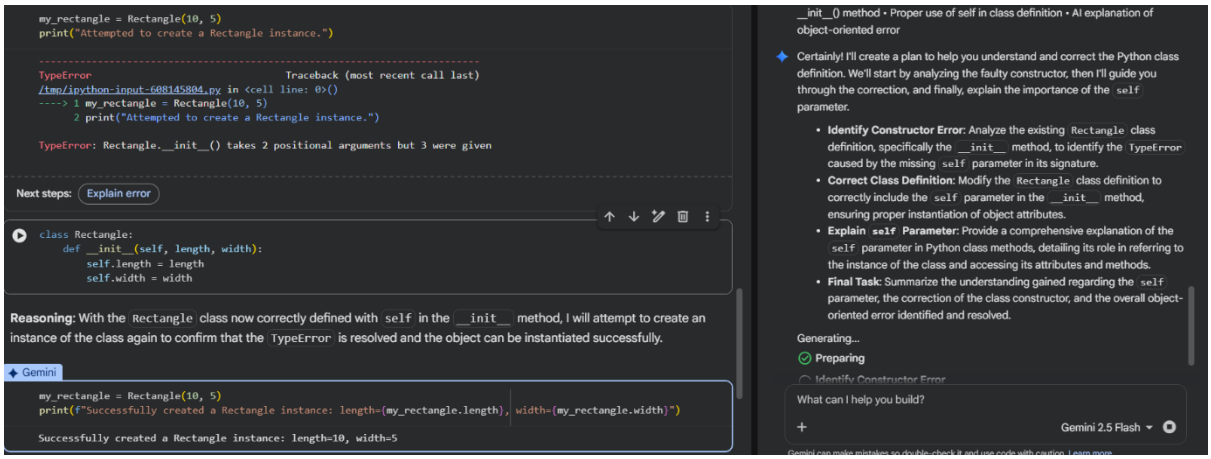
AI identifies the issue because self is required to reference instance variables within a class.

Without self, the object cannot store or access its attributes correctly.

AI Explanation:

AI corrects the constructor by adding the self parameter and properly initializing instance variables.

Expected Output Achieved
Corrected __init__() method
Proper use of self in class definition
AI explanation of object-oriented error



Task 5: Resolving Index Errors in Lists

```python
numbers = [1, 2, 3]
print(numbers[5])
```

Scenario

A program crashes when accessing an invalid index in a list.

AI Detection:

AI identifies an IndexError because the requested index exceeds the valid range of the list.

AI Explanation:

The AI suggests safe access techniques such as bounds checking or exception handling to prevent the crash.

These methods ensure that list elements are accessed only within valid limits.

Expected Output Achieved
Index error resolved
Safe list access logic implemented
AI suggestion using length checks or exception handling